DB

BASICS OF DATABASES

for Modern AI Applications

Practical foundations for AI engineers, data scientists & developers

Agenda & Learning Objectives

01

Introduction & Motivation

Why databases are critical in AI systems

02

Types of Databases for AI

Relational, Vector, Document, Graph, DuckDB, Parquet & Hybrid

03

Core Database Concepts

SQL, modeling, indexing, consistency

04

Typical AI Application Patterns

RAG, Feature Stores, Logging, Memory & more

01

Introduction & Motivation

Why databases are the foundation of every reliable AI system

Why Databases Matter in AI

Data is the fuel of AI. Databases are the engine that stores, organizes, retrieves and governs that fuel.

Training & Fine-tuning

Clean, versioned datasets for model training and continuous improvement

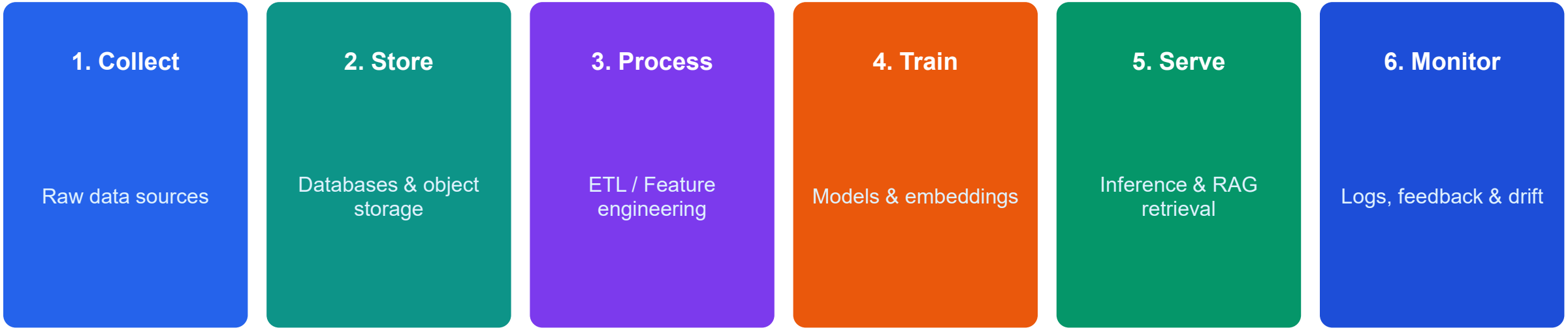
Retrieval & Context

Fast semantic search over documents, embeddings and knowledge bases (RAG)

Production Systems

Feature stores, logging, monitoring, user memory and experiment tracking

AI Data Lifecycle — Where Databases Fit



Key Insight

Modern AI systems are data pipelines. Databases appear at almost every stage — from storing raw training data and features, to holding embeddings for retrieval, to logging predictions and user feedback for continuous improvement.

What Goes Wrong Without Proper Databases

Hallucinations & Poor Context

LLMs without grounded retrieval invent facts. No vector store = weak RAG.

Data Drift & Stale Models

Without versioned feature stores and monitoring tables, models degrade silently.

Unreliable Production

No transactional storage for orders, users or state leads to inconsistent AI behavior.

Impossible Debugging

Missing prediction logs and input snapshots make root-cause analysis extremely hard.

Security & Compliance Gaps

Unstructured files scattered everywhere make access control and audits painful.

Scalability Collapse

Ad-hoc CSV/JSON files break under concurrent users and growing data volumes.

02

Types of Databases Relevant to AI

Choosing the right storage for structured data, embeddings, documents and relationships

Database Families for AI Applications

Relational

PostgreSQL, MySQL

- Structured tables & SQL
- ACID transactions
- Strong consistency
- Features, logs, metadata, CRM

Vector

pgvector, Pinecone, Weaviate

- Store embeddings
- Similarity search
- Core of RAG systems
- Often combined with Postgres

Document / NoSQL

MongoDB, Firestore

- Flexible JSON schemas
- Rapid prototyping
- Semi-structured data
- Content & configs

Graph

Neo4j, Amazon Neptune

- Entities + relationships
- Knowledge graphs
- Multi-hop reasoning
- Recommendation engines

Typical Scenarios by Database Type

Type	Best For	Classic AI / App Examples	When to Prefer
Relational	Structured data + relationships + ACID	CRM, orders, users, feature metadata, logs	Need joins, transactions, integrity
Document / NoSQL	Flexible / evolving schemas, JSON	Content stores, configs, event payloads	Schema changes often, rapid prototyping
Vector	Embeddings + similarity search	RAG, semantic search, recommendations	Semantic retrieval is core
Graph	Entities + complex relationships	Knowledge graphs, multi-hop reasoning, fraud	Path / relationship queries matter
DuckDB	Fast analytics on local / file data	Feature engineering, querying Parquet	In-process analytics, no server needed
Parquet	Efficient columnar storage of large data	Training sets, features, embeddings, logs	Large analytical datasets / data lakes

Relational Databases — Typical Scenarios

Best for structured data with clear relationships and the need for integrity (ACID).

Classic Example: CRM Systems

Customers, Contacts, Accounts, Orders, Support Tickets, Interactions — all linked by foreign keys.

- One customer → many orders / tickets / interactions
- Referential integrity (no orphan records)
- Complex joins for reporting & dashboards
- Transactional updates (status, ownership)
- Audit trails and access control

Other High-Value Scenarios

Transactional systems

Orders, payments, subscriptions, inventory

Feature store metadata

Online + offline feature definitions & versions

Prediction & experiment logs

Inputs, outputs, latency, feedback for audit

User & permission models

Accounts, roles, access control lists

Model registry metadata

Versions, metrics, lineage, artifacts links

Primary Key vs UNIQUE — the database checks uniqueness

You do not scan the table yourself. SQLite rejects a duplicate insert.

PRIMARY KEY

Identifies the row. Always unique. Never NULL.
One per table.
Example: instrument_id, bar_id

UNIQUE constraint

Extra business rule: this value must not repeat.
Can sit on one column or several together.
Example: symbol | (instrument_id, trade_date)

On the stock schema

```
instruments
  instrument_id   PRIMARY KEY
  symbol          UNIQUE          ← one row per ticker
```

```
daily_bars
  bar_id                          PRIMARY KEY
  UNIQUE (instrument_id, trade_date)
  ← one bar per stock per day
```

What the database allows

instrument_id	trade_date	Meaning	Result
1 (0700.HK)	2026-08-26	First bar for Tencent that day	OK
1 (0700.HK)	2026-08-25	Same stock, different date	OK
2 (9988.HK)	2026-08-26	Different stock, same date	OK
1 (0700.HK)	2026-08-26	Same stock + same date again	REJECTED

Example: Schema Design — Stock Screening for Algo Trading

instruments	daily_bars	screen_rules	screen_results
instrument_id (PK)	bar_id (PK)	rule_id (PK)	result_id (PK)
symbol (UNIQUE)	instrument_id (FK)	rule_name	rule_id (FK)
exchange	trade_date	metric	instrument_id (FK)
sector	close	operator	screen_date
listing_status	volume	threshold	metric_value
listed_at	avg_volume_20d	is_active	passed

Relationships: instruments 1——N daily_bars | instruments 1——N screen_results | screen_rules 1——N screen_results

Sample rule SQL · close > 50 AND volume > avg_volume_20d

```
SELECT i.symbol, b.trade_date, b.close, b.volume
FROM instruments i
JOIN daily_bars b ON b.instrument_id = i.instrument_id
WHERE b.trade_date = DATE('now')
  AND b.close > 50
  AND b.volume > b.avg_volume_20d;
```

Document / NoSQL — Typical Scenarios

Best when the data shape varies or evolves quickly and strict schemas would slow you down.

Content & knowledge bases

Articles, product catalogs with varying attributes, support articles — fields differ per document.

User profiles & configs

Profiles that accumulate new fields over time; application configuration documents.

Event / activity streams

Flexible event payloads where each event type can carry different properties.

Rapid prototyping

Early-stage products where the schema is still changing weekly.

Semi-structured logs

Application or agent logs that do not fit a rigid relational model.

Hybrid with relational

Core entities in Postgres; flexible content or activity data in MongoDB / Firestore.

Vector Databases — Typical Scenarios

Core technology for semantic retrieval. Almost every modern AI application that needs "search by meaning" uses vectors.

Retrieval-Augmented Generation (RAG)

The dominant use case. Embed documents → store vectors + metadata → retrieve top-k chunks at query time → feed to LLM.

Semantic search

Search across support tickets, code, product descriptions, internal knowledge by meaning instead of keywords.

Recommendations

User and item embeddings for "similar products", "users like you also..." style recommendations.

Multimodal similarity

Text ↔ image, audio embeddings, or cross-modal retrieval.

Agent memory

Long-term semantic memory for agents — retrieve relevant past interactions or facts by meaning.

Hybrid search

Combine vector similarity with keyword filters and metadata (source, date, access rights).

Embedding and Similarity Comparison Vector DB

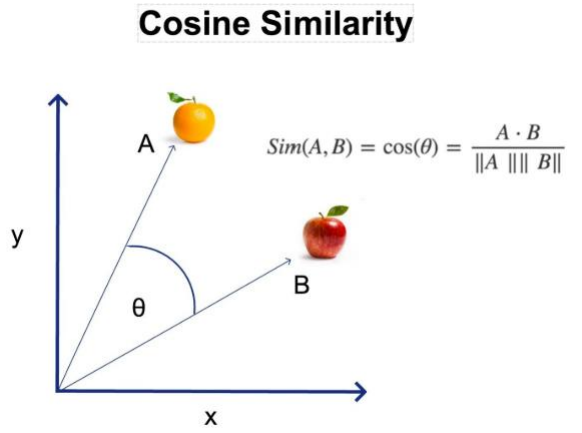
Apple

Orange

vector

```
-0.01871234,  0.03124567, -0.00782345,  0.04415678, -0.02133456,
 0.04151234, -0.00391234,  0.06342310, -0.00123456,  0.08567890,
-0.05123456,  0.01782345, -0.07234567,  0.01567890, -0.03678901,
 0.06123456, -0.01894567,  0.04678901, -0.08345678,  0.00912345,
 0.03456789, -0.01098765,  0.06901234, -0.02789012,  0.01543210,
 0.05234567, -0.02210987,  0.04198765, -0.07890123,  0.00678901,
... (continues exactly 1536 values) ...
-0.03912345,  0.02678901, -0.05432109,  0.03210987, -0.01456789,
-0.04210987,  0.02345678, -0.05789012,  0.02987654, -0.01345678
```

```
-0.01984567,  0.03256789, -0.00691234,  0.04578901, -0.02012345,
 0.04298765, -0.00278901,  0.06512345, -0.00098765,  0.08734567,
-0.04987654,  0.01898765, -0.07012345,  0.01678901, -0.03545678,
 0.06345678, -0.01765432,  0.04812345, -0.08123456,  0.01034567,
 0.03389012, -0.01123456,  0.06890123, -0.02678901,  0.01654321,
 0.05123456, -0.02198765,  0.04321098, -0.07654321,  0.00789012,
... (exactly 1536 values total) ...
-0.04098765,  0.02567890, -0.05678901,  0.03123456, -0.01234567,
-0.04123456,  0.02456789, -0.05567890,  0.03098765, -0.01189012
```



Similarity comparison
0.842

Similarity from 0 to 1

Graph Databases — Typical Scenarios

Choose a graph database when the relationships between entities are as important as the entities themselves.

Knowledge graphs for reasoning

Multi-hop questions. Enables richer agent reasoning over connected facts.

Recommendation graphs

User–item–context graphs where paths reveal non-obvious recommendations.

Fraud & risk networks

Detect rings, shared devices, unusual relationship patterns that tabular data hides.

Entity resolution & identity

Linking people, companies, devices across systems via relationship strength.

Organizational & process maps

Teams, ownership, dependencies — useful for AI-assisted impact analysis.

When NOT to use

Simple lookups, bulk analytics, or pure RAG retrieval — relational + vector is usually simpler.

DuckDB & Parquet — Typical Scenarios

The modern analytical layer for AI data work. Not a replacement for Postgres, but an essential complement.

DuckDB

In-process analytical SQL engine

- Query Parquet / CSV files directly — no import step
- Feature engineering & large-scale data exploration
- Local / embedded analytics inside Python pipelines
- Fast OLAP-style aggregations on historical data
- Excellent for backtesting and offline evaluation
- Zero server management; runs in your process

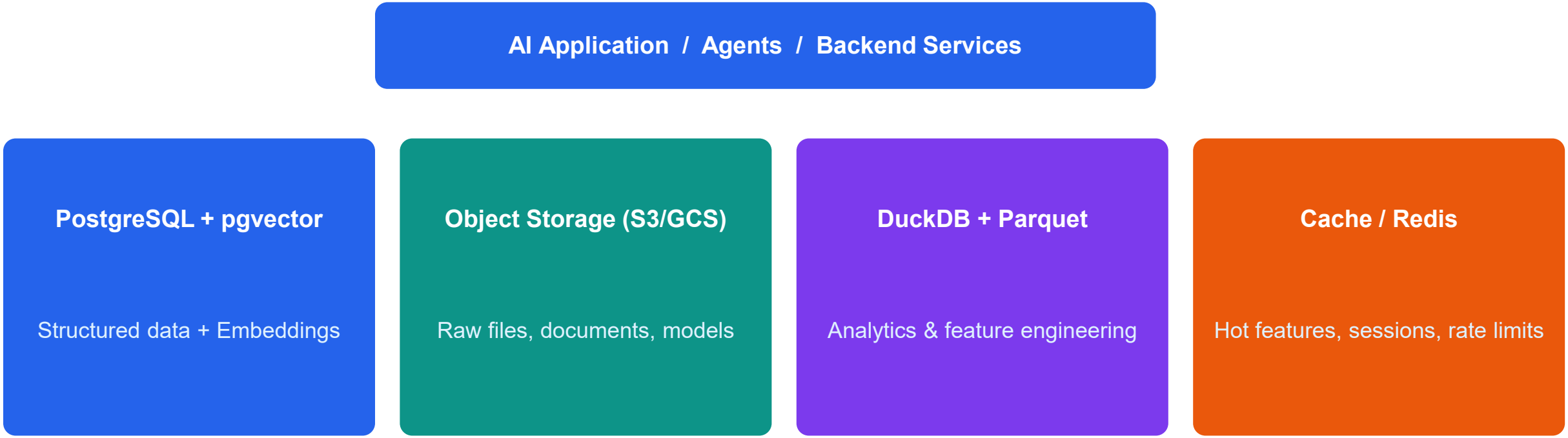
Parquet

Columnar file format for large datasets

- Training datasets & feature matrices
- Embedding archives and prediction logs
- Data-lake style storage (S3 / local)
- Excellent compression + selective column reads
- Native support in DuckDB, Polars, Pandas, Spark
- Ideal for versioned offline feature stores

The Modern Reality: Hybrid Stacks

Most production AI systems combine multiple database types. One practical pattern:



Practical tip: Start with PostgreSQL + pgvector (or Supabase / Neon). Add DuckDB + Parquet for analytics. It covers the majority of AI application needs without managing many systems.

Practical Cloud Options for AI Applications

Supabase

Best starting point for most AI projects

What it is

Managed Postgres + Auth + Storage + Realtime + Edge Functions

AI strengths

pgvector built-in, easy Python client, free tier, Row Level Security

Best for

RAG prototypes, internal tools, startups, personal projects, MVPs

Watch out

Free projects pause after inactivity; scale limits on free tier

AWS

Enterprise scale & full AI platform

Key services

RDS / Aurora (Postgres), OpenSearch, DynamoDB, S3, Bedrock

AI strengths

Bedrock Knowledge Bases, managed vector search, high scale & compliance

Best for

Production systems, regulated industries, large data, complex pipelines

Watch out

Steeper learning curve and higher cost if not carefully designed

Quick Decision Guide

Choose the right starting point based on your stage and constraints

Learning / Prototype

- Supabase or Neon
- Local Postgres + pgvector
- Chroma / simple vector DB
- DuckDB + local Parquet

MVP / Small Team

- Supabase (full backend)
- Neon + Vercel / Railway
- Pinecone for vectors only
- Parquet on S3 + DuckDB

Production / Scale

- AWS Aurora + OpenSearch
- Supabase Pro / Enterprise
- Dedicated vector DB + RDS
- Feature store + object storage

Enterprise / Regulated

- AWS (RDS, Bedrock, IAM)
- Self-hosted on VPC
- Strong audit & compliance focus
- Data lineage & access controls

03

Core Database Concepts for AI Practitioners

The essential knowledge every AI engineer should have

Essential Building Blocks

Tables & Schema

Rows, columns, data types. Design tables for entities: users, documents, embeddings, predictions.

Keys & Relationships

Primary keys uniquely identify rows. Foreign keys link tables (e.g. document → chunks).

SQL Basics

SELECT, INSERT, UPDATE, DELETE, JOIN, WHERE, ORDER BY, LIMIT — enough for most AI work.

Indexing

Speeds up lookups. Critical for vector indexes (HNSW/IVFFlat) and frequent filter columns.

Consistency & ACID

Transactions protect integrity. Important when writing features + logs together.

Data Modeling for AI

Separate structured metadata from embeddings. Version datasets and features explicitly.

Example: Tables & Schema Design — Customer CRM

customers	orders	support_tickets	interactions
customer_id (PK)	order_id (PK)	ticket_id (PK)	interaction_id (PK)
full_name	customer_id (FK)	customer_id (FK)	customer_id (FK)
email (UNIQUE)	order_date	subject	channel
phone	total_amount	priority	summary
created_at	status	status	agent_id
status	channel	opened_at	created_at

Relationships: customers 1——N orders | customers 1——N support_tickets | customers 1——N interactions

One customer can have many orders, tickets, and interactions. Foreign keys enforce referential integrity.

Example Data Model for a RAG Application

documents	chunks	queries_log
id (PK)	id (PK)	id (PK)
title	document_id (FK)	user_id
source	content	question
created_at	embedding (vector)	retrieved_ids
metadata (JSONB)	token_count	answer
		latency_ms

documents 1——N chunks | queries_log stores every RAG interaction for analysis

04

Typical AI Application Patterns

How databases are actually used in real AI systems today

Pattern 1: Retrieval-Augmented Generation (RAG)

1

Ingest

Split docs → embed → store vectors + metadata

2

Query

User question → embed → similarity search

3

Retrieve

Top-k chunks returned from vector DB

4

Generate

LLM answers with retrieved context

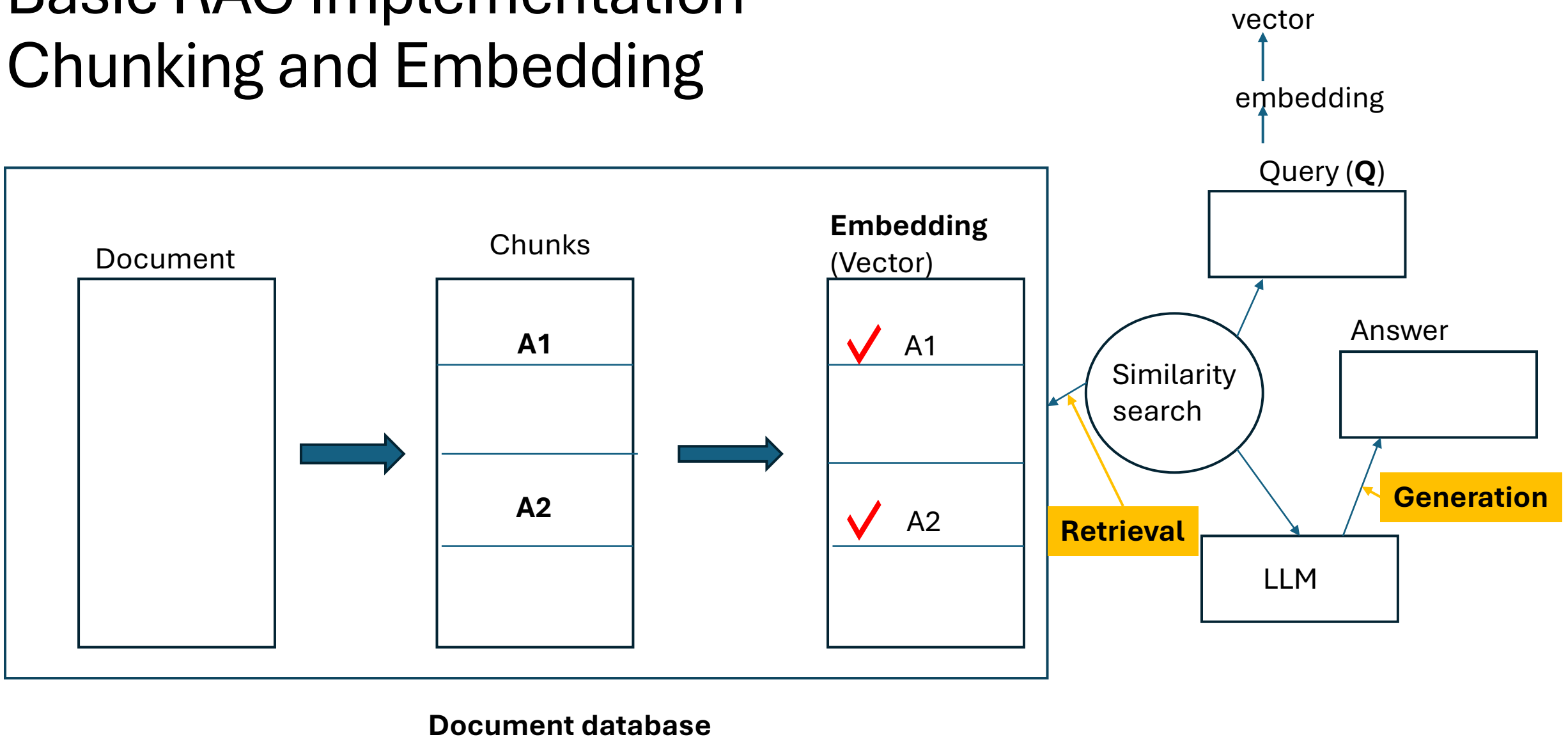
Database Role in RAG

- Store document chunks + metadata
- Store embedding vectors (pgvector / dedicated)
- Support fast similarity search (cosine / L2)
- Filter by metadata (source, date, access)
- Log every query + retrieved context

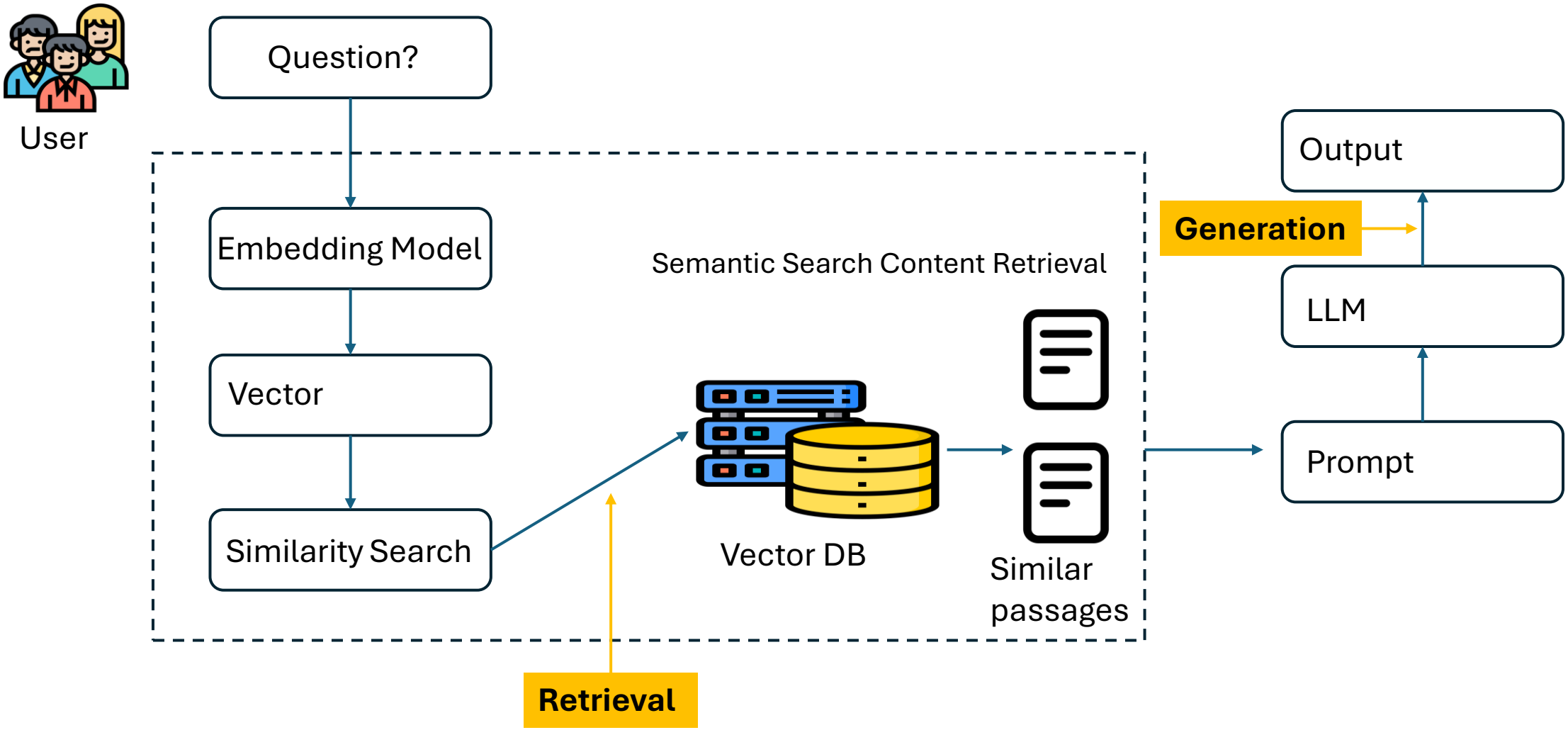
Common Tools

- Postgres + pgvector (simple & powerful)
- Supabase / Neon (managed Postgres)
- Pinecone, Weaviate, Chroma, Milvus
- LangChain / LlamaIndex integrations
- Hybrid search (keyword + vector)

Basic RAG Implementation
Chunking and Embedding



Basic RAG Implementation



Other High-Value AI Database Patterns

Feature Store

Store online & offline features for models
Feast, Tecton, custom Postgres

Model Registry

Track models, versions, metrics, artifacts
MLflow + DB, custom tables

Prediction Logging

Inputs, outputs, latency, feedback for audit
Postgres, ClickHouse

Chat / Agent Memory

Conversation history & long-term memory
Postgres, Redis + Postgres

Recommendation

User-item interactions + embeddings
Hybrid relational + vector

Training Data Mgmt

Labeled datasets, versions, splits
Postgres + object storage / Parquet

Key Takeaways

1. Databases are not optional in serious AI systems — they power training, retrieval, serving and monitoring.

2. Know the families: Relational, Vector, Document, Graph, plus DuckDB & Parquet for analytics. Most teams start with Postgres + vectors.

3. Choose by access pattern: CRM-style structured data → Relational; semantic search → Vector; flexible content → NoSQL; relationships → Graph; local analytics → DuckDB + Parquet.

4. Master basic SQL, schema design, keys and indexing. These skills transfer across tools.

5. RAG, Feature Stores, Logging and Memory are the highest-ROI patterns to learn first.

6. Hybrid is normal. Start simple (Postgres + pgvector + Parquet), then add specialized stores only when needed.

Thank you

Questions & Discussion

Start simple. Postgres + pgvector + Parquet covers most AI needs.
Add specialized databases only when the access pattern demands it.